```
def process(session: S, envelope: Envelope): Unit
```

where `Envelope` is a wrapper containing the event retrieved from the journal, like `EventEnvelope[SPContainer.Event]`, and `S` is a subtype of `JdbcSession`, like `ScalikeJdbcSession`.

Listing 10.6 Extension of `JdbcHandler`

```
import example.repository.scalike.{
  CargosPerContainerRepository,
  ScalikeJdbcSession
}
import akka.projection.jdbc.scaladsl.JdbcHandler
import akka.projection.eventsourced.EventEnvelope

import example.persistence.SPContainer

class CPCProjectionHandler(
    repository: CargosPerContainerRepository)              ❶
    extends JdbcHandler[
      EventEnvelope[SPContainer.Event],
      ScalikeJdbcSession] {

  val logger = LoggerFactory.getLogger(classOf[CPCProjectionHandler])

  override def process(                                    ❷
      session: ScalikeJdbcSession,
      envelope: EventEnvelope[SPContainer.Event]): Unit = {
    envelope.event match {
      case SPContainer.CargoAdded(containerId, cargo) =>
        repository.addCargo(containerId, session)          ❸
      case x =>
        logger.debug("ignoring event {} in projection",x)
    }
  }
}
```

❶ Passes through the repository

❷ Overrides the JdbcHandler process method

❸ Updates the database when processing CargoAdded events

This cargo-per-container projection handler stores the events by grouping the sum of `CargoAdded` by container. This is accomplished by a repository implementation that converts the `CargoAdded` envelope to an upsert in the database. This database statement creates the tuple `([containerId], 1)` the first time; otherwise, it adds one to the existing tuple.

Listing 10.7 Repository Implementation

```
trait CargosPerContainerRepository {                       ❶

  def addCargo(containerId: String, session: ScalikeJdbcSession): Unit
}

class CargosPerContainerRepositoryImpl extends CargosPerContainerRepository {

  override def addCargo(
      containerId: String,
      session: ScalikeJdbcSession): Unit = {
    session.db.withinTx { implicit dbSession =>            ❷
      sql"""INSERT INTO cargos_per_container (containerId, carg   ❸
              VALUES ($containerId, 1)
                ON CONFLICT (containerId) DO
              UPDATE SET cargos = cargos_per_container.cargos + 1
```